

EV Battery Health Digital Twin & Prognostics

Detailed Project Blueprint — Problem Definition, Datasets, ML Architecture, Digital Twin, What-If Simulation, GenAI Copilot, Evaluation & Implementation Roadmap

Recommended positioning: A data-driven lithium-ion battery digital twin for state estimation, degradation forecasting, remaining useful life prediction, anomaly detection, scenario simulation, and engineer-facing GenAI assistance.

Primary dataset	NASA Li-ion Battery Aging Dataset
Validation dataset	Oxford Battery Degradation Dataset 1
Optional extension	CALCE Battery Research Data
Primary ML problems	SOH estimation + RUL prediction + anomaly detection
Advanced layer	Probabilistic prognostics + counterfactual/what-if simulation
GenAI layer	Tool-using Battery Engineer Copilot

1. Executive Summary

The project builds a virtual representation of a lithium-ion battery whose internal state is continuously inferred from observed telemetry. The system should estimate current health, learn how degradation is progressing, forecast when the battery will cross an end-of-life threshold, detect abnormal degradation, quantify uncertainty, and expose the results through a practical engineer-facing interface.

The core design principle is to keep numerical computation deterministic and model-driven. The GenAI component should sit on top of validated analytical tools rather than inventing battery measurements or predictions. This makes the project credible as an AI/DS engineering system rather than a generic chatbot.

Recommended final framing: “EV Battery Health Digital Twin & Prognostics Platform” — a cell-level digital twin architecture that can be extended to a simulated battery pack.

2. Detailed Problem Statement

Business/engineering problem: EV batteries degrade over time. Operators need to know the current health of a battery, how quickly it is degrading, when it may reach an operational end-of-life threshold, whether its degradation is abnormal, and which operating conditions are associated with faster deterioration.

Technical problem: Given multivariate, time-dependent measurements such as voltage, current, temperature, time, capacity and impedance, construct a virtual battery state that estimates SOH and forecasts future degradation and RUL with calibrated uncertainty.

Advanced problem: Use the learned degradation representation to answer controlled what-if questions, such as how a change in thermal exposure or load profile might alter the projected trajectory. These scenario outputs should be explicitly presented as model-based simulations, not automatically claimed to be causal conclusions.

3. Project Goals & Non-Goals

Goal	Definition of success
Battery state representation	A standardized cycle-level/temporal representation of battery operating and health signals.
SOH estimation	Estimate capacity-based health accurately from observable telemetry and derived features.
RUL prediction	Predict remaining cycles/time to a defined EOL threshold without future-data leakage.
Uncertainty	Return prediction intervals or calibrated uncertainty rather than a single unsupported number.
Anomaly detection	Identify batteries/cycles whose degradation behavior deviates from expected patterns.
Driver analysis	Quantify which signals are associated with model-predicted degradation using explainability methods.
Digital twin	Maintain a virtual battery state and projected degradation trajectory.
Scenario simulation	Evaluate controlled changes to operating conditions using an explicit simulation layer.
GenAI copilot	Translate validated model outputs into technical explanations and tool-driven answers.

Non-goals for the first version: claiming a physically exact electrochemical simulation; claiming arbitrary what-if scenarios are causal; building a production BMS; or using an LLM to calculate SOH/RUL.

4. Datasets

4.1 Primary: NASA Li-ion Battery Aging Dataset

NASA's Prognostics Center of Excellence battery repository provides lithium-ion battery aging data collected on a battery prognostics testbed. The data includes charge/discharge measurements and

impedance-related measurements across aging cycles. NASA's documentation defines an experimental end-of-life point based on capacity falling from a nominal 2 Ah to 1.4 Ah (30% capacity fade).

- Useful signals: voltage, current, temperature, time, capacity and impedance-related variables.
- Why it fits: the batteries are aged through repeated cycling, creating longitudinal degradation trajectories.
- Best first use: build the full data pipeline, SOH target, RUL target, baselines and first digital-twin state.
- Important caveat: these are individual cells/experimental batteries, not a production EV pack. The final project should say this clearly.

Source: NASA PCoE / NASA Li-ion Battery Aging Dataset.

4.2 Secondary: Oxford Battery Degradation Dataset 1

The Oxford dataset contains long-term ageing experiments on lithium-ion pouch cells. The cells were tested under a controlled temperature and cycling protocol, including an urban Artemis drive-cycle discharge profile, with periodic characterization measurements.

- Use it primarily for external validation/generalization rather than mixing it into the first training set.
- Compare whether features and degradation representations learned from NASA transfer to a different experimental setup.
- Report performance separately on NASA and Oxford; do not merge them blindly and call the result generalization.

4.3 Optional: CALCE Battery Research Data

CALCE provides open battery research data spanning different cycling profiles, storage conditions, chemistries and cell formats. It is useful as a later extension for testing whether the pipeline can handle more heterogeneous degradation behavior.

Recommended sequence: NASA → complete MVP → Oxford external validation → CALCE only if the core system is stable.

5. Data Model & Processing Pipeline

Raw experiment files should be converted into a canonical analytical schema. A useful first table is:

Field	Purpose
battery_id	Unique battery/cell identifier
cycle_id	Age/operating-cycle index
timestamp	Within-cycle temporal ordering
voltage_v	Electrical state
current_a	Load/charge stress
temperature_c	Thermal state
capacity_ah	Health target where measured
energy_wh	Usable energy proxy
internal_resistance_ohm	Degradation-related signal

Field	Purpose
charge_duration_s	Behavioral feature
discharge_duration_s	Behavioral feature
coulomb_throughput	Usage intensity

Processing steps

- Parse source files and normalize units.
- Segment measurements into charge/discharge cycles or equivalent operating windows.
- Perform quality checks: missingness, impossible values, duplicate timestamps, discontinuities and sensor spikes.
- Extract cycle-level features while retaining selected high-frequency curve information.
- Create battery-specific trajectories and preserve battery identity for split logic.
- Store both raw/cleaned telemetry and derived features so experiments remain reproducible.

6. Target Definitions

6.1 State of Health (SOH)

A simple capacity-based target is:

$$\text{SOH}(t) = Q(t) / Q(\text{initial}) \times 100$$

where $Q(t)$ is measured discharge capacity at cycle t and $Q(\text{initial})$ is the initial reference capacity. For NASA, align the primary EOL definition with the dataset's experimental 30% capacity-fade criterion.

6.2 Remaining Useful Life (RUL)

$$\text{RUL}(t) = N_{\text{EOL}} - N_t$$

If the current cycle is 420 and the EOL threshold is reached at cycle 610, the target RUL is 190 cycles. For every prediction point, only information available up to that point may be used.

6.3 Pack-level extension

After the cell-level system is stable, simulate a pack from multiple virtual cells. Track cell-to-cell SOH dispersion and define pack-level health around the weakest cell or another explicitly documented pack criterion.

7. Modeling Strategy

Stage A — Baselines

- Linear regression / polynomial regression for simple degradation curves.
- Random Forest or XGBoost using cycle-level engineered features.
- A naive persistence/last-value baseline for SOH and a simple degradation-curve baseline for RUL.

Stage B — Temporal modeling

- LSTM/GRU, Temporal Convolutional Network, or another sequence model.
- Input a rolling window of recent cycles rather than treating each cycle independently.

- Compare against tree-based models to determine whether temporal complexity actually helps.

Stage C — Probabilistic prognostics

- Quantile regression for lower/median/upper RUL estimates.
- Bootstrap ensembles or deep ensembles for uncertainty.
- Conformal prediction as a later calibration layer for prediction intervals.

Recommended output for each battery state:

SOH = 82.7% | RUL = 146 cycles | 90% interval = 123-179 | degradation trend = accelerating | confidence = calibrated score

8. Avoiding Data Leakage

Battery prognostics is especially vulnerable to leakage. The most important rule is: a prediction at cycle t must be computed using information that would actually have been available by cycle t .

- Do not random-split individual rows/cycles when evaluating generalization across batteries; this can place nearly identical trajectories in train and test.
- Prefer battery-level splits for cross-battery generalization.
- Use temporal holdouts when evaluating early-life prediction of future degradation.
- Do not calculate normalization statistics using future test data.
- Do not construct features using future cycles, future EOL, or complete-battery statistics unavailable at inference time.
- Keep an explicit experiment manifest containing train batteries, validation batteries, test batteries and prediction horizon.

A strong evaluation design uses two views:

- Within-battery forecasting: early cycles → later cycles.
- Cross-battery generalization: unseen battery IDs → full future trajectory.

9. Degradation Analysis & Anomaly Detection

Build a peer-group baseline for expected degradation. A battery becomes interesting when its residual trajectory, degradation rate or thermal/electrical behavior deviates materially from peers.

- Isolation Forest or robust statistical residual scoring as a first baseline.
- Autoencoder/sequence model as an advanced option.
- Change-point detection to identify the beginning of accelerated degradation.
- Measure detection lead time: how early an abnormal battery is identified before EOL or a predefined risk point.

Do not assume every anomaly is a failure. Label it as an abnormality requiring investigation unless the dataset provides a defensible failure mechanism.

10. Degradation Driver Analysis

Use SHAP, permutation importance and controlled analyses to understand which features drive model predictions. Candidate drivers include thermal exposure, current stress, voltage behavior, cycle count, throughput and impedance.

Important scientific distinction: model feature importance is not causal proof. Phrase results as “signals associated with model-predicted degradation” unless you have a suitable causal design.

11. Digital Twin Design

The digital twin is a structured virtual state, not merely a dashboard. At any point in time it should maintain:

Twin state	Example
Battery identity	B0005
Current age	420 cycles
Estimated SOH	82.7%
Estimated RUL	146 cycles
RUL uncertainty	123–179 cycles (90% interval)
Degradation rate	–0.041 percentage points/cycle
Thermal state	Medium/high exposure
Anomaly score	0.81
Health trend	Accelerating
Model version	rul_model_2026_09

Persisting model version and confidence with the state is important: a digital twin should be auditable, not just a pretty visualization.

12. What-If Simulation

The scenario engine should take a current battery state and modify selected operating assumptions, then project an alternative trajectory through the degradation model.

- Temperature scenario: e.g., +5°C average thermal exposure.
- Load/current scenario: e.g., +20% average discharge current.
- Cycling scenario: shallow vs. deeper cycling assumptions.
- Charging scenario: alternative charging intensity where supported by the available data.
- Compare baseline and scenario SOH/RUL trajectories.

Recommended output: baseline RUL 210 cycles versus scenario RUL 165 cycles, with an uncertainty band and a clear note that the result is model-based scenario simulation.

If the training data does not contain sufficient variation for a scenario, the system should say so rather than extrapolate confidently. This is an important part of a credible digital twin.

13. GenAI / Battery Engineer Copilot

The GenAI layer should not calculate electrical quantities itself. It should call deterministic tools that return structured results, then explain those results in engineering language.

Suggested tools

- `get_battery_state(battery_id)`
- `get_soh(battery_id, cycle)`
- `predict_rul(battery_id, cycle)`
- `get_degradation_drivers(battery_id, cycle)`
- `detect_anomaly(battery_id, cycle)`
- `compare_batteries(battery_a, battery_b)`
- `simulate_scenario(battery_id, temperature_delta, load_multiplier, cycling_profile)`
- `get_model_metadata(model_version)`

Example questions

- What is the current health of B0005?
- Why is B0005 degrading faster than B0006?
- How reliable is the RUL estimate?
- Is this battery showing abnormal behavior?
- What are the strongest model-associated degradation signals?
- Simulate +5°C thermal exposure and compare the projected trajectory.
- Compare the health trajectory of two batteries.

For an advanced implementation, expose the tools through an MCP-style interface so the copilot is clearly separated from the analytical services.

14. System Architecture

Recommended logical architecture:

Raw battery files → ingestion/cleaning → cycle segmentation → feature engineering → feature store → SOH/RUL/anomaly models → digital-twin state service → scenario simulator → tool layer → GenAI copilot/UI

Suggested technology stack

Layer	Suggested choices
Data processing	Python, Pandas/Polars, NumPy; Spark only if needed
Modeling	scikit-learn, XGBoost/LightGBM, PyTorch
Experiment tracking	MLflow
Data validation	Great Expectations or custom validation checks
Serving	FastAPI
Storage	Parquet + DuckDB/PostgreSQL for an MVP
Vector/RAG	Only if you add technical-document retrieval; not required for core ML
Agent layer	Tool calling / MCP-compatible service
UI	Streamlit or a lightweight web app

Layer	Suggested choices
Deployment	Docker; cloud deployment only after local MVP works

15. Evaluation Framework

Component	Primary metrics
SOH	MAE, RMSE, R ² ; error by battery age
RUL	MAE/RMSE in cycles; error by prediction horizon
Uncertainty	Coverage probability, interval width, calibration
Anomaly detection	Precision, recall, F1, detection lead time
Cross-battery generalization	Metrics on completely unseen battery IDs
Scenario simulation	Validation against held-out conditions where available; otherwise uncertainty + extrapolation checks
Copilot	Tool-call correctness, factual consistency, groundedness, refusal/uncertainty behavior

Avoid presenting a single accuracy number. Show performance as a function of battery age and prediction horizon, because early-life RUL prediction is materially harder than near-EOL prediction.

16. Implementation Roadmap

Phase	Work	Deliverable
1	Acquire NASA data; parse files; understand experiments	Data dictionary + raw-to-clean pipeline
2	Cycle segmentation + feature extraction + EDA	Canonical battery dataset + degradation plots
3	SOH baselines + XGBoost/LightGBM	SOH model + evaluation report
4	Temporal RUL modeling	RUL model + temporal holdout evaluation
5	Uncertainty + calibration	Probabilistic RUL API
6	Anomaly/change-point detection	Battery risk monitor
7	Digital twin state service	Versioned virtual battery state
8	Scenario engine	What-if simulator
9	Oxford external validation	Cross-dataset evaluation
10	Tool layer + GenAI copilot	Engineer-facing agent
11	UI + deployment + documentation	Portfolio-ready application

17. Recommended Repository Structure

A clean repository structure will make the project read like an engineering system rather than a collection of notebooks.

```
ev-battery-digital-twin/ ├── data_contracts/ ├── configs/ ├── src/ | ├── ingestion/ | ├── preprocessing/ | ├──
feature_engineering/ | ├── soh/ | ├── rul/ | ├── anomaly/ | ├── simulation/ | ├── twin/ | ├── api/ | └── agent/ ├──
notebooks/ | ├── 01_data_exploration/ | ├── 02_soh_baselines/ | ├── 03_rul/ | └── 04_external_validation/ ├── tests/
├── dashboards/ ├── models/ ├── reports/ ├── docker/ ├── requirements.txt └── README.md
```

18. Strong Experiments to Include

- Experiment 1: Does engineered cycle-level data outperform cycle-count-only degradation curves?

- Experiment 2: Do temporal models outperform XGBoost at early prediction horizons?
- Experiment 3: How does RUL error change when only the first 20%, 40%, 60% of battery life is observed?
- Experiment 4: Does an uncertainty-aware model remain calibrated across different batteries?
- Experiment 5: Can a model trained on NASA generalize to Oxford without retraining?
- Experiment 6: Can abnormal degradation be detected early enough to be operationally useful?
- Experiment 7: Which telemetry signals consistently contribute to degradation predictions?
- Experiment 8: How much value does the GenAI copilot add over a standard dashboard?

19. Portfolio / Resume Positioning

Avoid a resume bullet such as “Built an LSTM model to predict battery life.” It undersells the project.

Better positioning: “Built a data-driven lithium-ion battery digital twin using longitudinal telemetry to estimate SOH, forecast probabilistic RUL, detect accelerated degradation and expose validated battery-state tools through an agentic GenAI copilot; evaluated cross-battery and cross-dataset generalization.”

A second bullet can emphasize engineering: “Designed leakage-safe temporal evaluation, model versioning, uncertainty calibration, scenario simulation and an API/tool layer for reproducible battery prognostics.”

20. Key Risks & How to Handle Them

Risk	Mitigation
Dataset is cell-level, not EV-pack-level	State the limitation; build a documented virtual pack extension rather than claiming real pack meas
Very small number of batteries	Use battery-level cross-validation carefully; avoid overclaiming deep-model generalization.
Leakage through random splits	Use battery-level and temporal holdouts.
LLM hallucination	LLM can only answer from structured tool outputs and retrieved technical docs.
Overconfident what-if claims	Flag extrapolation and report scenario uncertainty.
Overengineering too early	Build NASA-only MVP before Oxford/CALCE, agent, deployment and pack simulation.
Metrics look good only near EOL	Report performance by prediction horizon and observed-life fraction.

21. Definition of Done

- A reproducible pipeline converts public battery data into a canonical analytical dataset.
- SOH is defined mathematically and predicted with leakage-safe evaluation.
- RUL is predicted from partial trajectories, not from information from the future.
- Prediction intervals are evaluated for coverage and calibration.
- Anomaly detection identifies accelerated or unusual degradation with measurable lead time.
- A versioned digital-twin state can be queried through an API.
- A scenario engine clearly distinguishes interpolation/validated scenarios from uncertain extrapolation.
- Oxford data provides a genuine external validation experiment.

- The GenAI copilot uses tools and grounded outputs rather than generating battery measurements itself.
- The repository includes tests, experiment configurations, model metadata and a clear README.

22. Primary References & Dataset Links

NASA PCoE / Li-ion Battery Aging Dataset: NASA data repository and battery aging documentation. Use the official NASA repository as the canonical source for acquisition and dataset description.

Oxford Battery Degradation Dataset 1: University of Oxford ORA repository. Use the repository record for dataset details and access conditions.

CALCE Battery Research Data: University of Maryland CALCE. Use the official CALCE battery data page for available datasets and experimental descriptions.

Important: Dataset availability, access procedures and repository URLs can change. The project should record the exact dataset version/download date in its experiment manifest.

23. Recommended First Week

- Day 1: Download NASA data and inspect the file structure and experiment metadata.
- Day 2: Build a parser and produce one clean battery trajectory.
- Day 3: Segment charge/discharge cycles and calculate capacity/SOH.
- Day 4: Plot degradation trajectories and compare batteries.
- Day 5: Build a naive SOH baseline and leakage-safe train/test split.
- Weekend: Write the data dictionary, experiment protocol and first results report before adding complex models.

Final recommendation: Treat this as a 3-layer project: (1) rigorous battery prognostics, (2) digital-twin state + scenario simulation, and (3) GenAI engineer interface. The first layer is what makes the project scientifically credible; the second makes it distinctive; the third demonstrates your GenAI/agent-AI transition.